

Programmation fonctionnelle 2
Preuves et programmes
CM3 : Tuples, Curryfication, False

Pierre Rousselin

Université Sorbonne Paris Nord

février 2026

Tactiques et règles de déduction naturelle

On continue à avancer dans :

connecteur	introduction (prouver)	élimination (utiliser)
$\rightarrow$	intros	apply
$\wedge$	split	destruct
$\vee$	left ou right	destruct
$\forall$	intros	specialize , instanciation ¹
$\perp$		destruct
$\exists$	exists	destruct

Dans chaque cas, il s'agit, en fait :

- ▶ de définition (**intros**) de fonctions ou
- ▶ d'application (**apply**) de fonctions ou
- ▶ d'élimination (**destruct**) d'un terme dont le type est inductif (donc en fait **match**) ou
- ▶ de construction (**split**, **left**, **right**) d'un élément d'un type inductif avec un de ses constructeurs appliqué à des arguments du bon type

1. explicite ou implicite avec unification

Types de fonctions en Rocq et en OCaml

Tuples en OCaml et en Rocq

Curryfication

Plusieurs flèches dans Prop

False et négation

Type fonction en OCaml : premiers exemples

- Rappel : en programmation fonctionnelle, les fonctions sont des valeurs comme les autres.
- Attacher une « valeur fonction » à un nom :

```
let incr = fun x -> x + 1  
(* val incr : int -> int = <fun> *)
```

- L'inférence de type a permis de connaître le type du résultat et de l'argument, mais si on veut on peut le préciser :

```
let incr = fun (x : int) -> (x + 1 : int)  
(* val incr : int -> int = <fun> *)
```

- Idem avec sucre syntaxique, on met une variable avant le symbole = qui attache

```
let foo b = if b then 42 else 2  
(* val foo : bool -> int = <fun> *)
```

- Bref, si t_1 et t_2 sont des types, $t_1 \rightarrow t_2$ est le type des fonctions de a vers b

Type fonction en Rocq : premiers exemples

- Attacher une « valeur fonction » à un nom :

```
Definition incr := fun x => x + 1.  
(* incr : nat -> nat *)
```

- Idem avec sucre syntaxique, on met une variable avant le signe := qui attache

```
Definition incr' x := x + 1.  
(* is_zero : int -> int *)  
Print incr'. (* is_zero = fun x => x + 1 *)
```

- Definition foo (b : bool) := if b then 42 else 2.
Check foo. (* foo : bool -> nat *)

- Là aussi, si t_1 et t_2 sont des types, $t_1 \rightarrow t_2$ est le type des fonctions de t_1 vers t_2 .

Et les fonctions de plusieurs variables ?

- ▶ *Shocking truth* : les fonctions de plusieurs variables n'existent pas.
- ▶ (mais pas d'inquiétude, on a mieux que ça)
- ▶ stratégie 1 (bof) : une fonction qui prend un argument dont le type est un tuple
- ▶ stratégie 2 (tip top) : curryfication, une fonction qui retourne une fonction

Types de fonctions en Rocq et en OCaml

Tuples en OCaml et en Rocq

Curryfication

Plusieurs flèches dans Prop

False et négation

Tuples en OCaml (1)

```
(* définition d'un triplet d'int *)
let t = (12, 13, 1);;
(* val t : int * int * int = (12, 13, 1) *)
(* élimination avec match : *)
let s = match t with (a, b, c) -> a + b + c;;
(* val s : int = 26 *)
(* élimination avec motif irréfutable : *)
let s = let (a, b, c) = t in a + b + c;;
(* val s : int = 26 *)
```


Tuples en OCaml (2)

Définir une fonction qui prend un tuple en argument :

```
let add_tuple2 c = match c with (x, y) -> x + y
(* val add_tuple2 : int * int -> int = <fun> *)
let add_tuple2 c = let (x, y) = c in x + y
(* val add_tuple2 : int * int -> int = <fun> *)
let add_tuple2 (x, y) = x + y
(* val add_tuple2 : int * int -> int = <fun> *)
let add_tuple3 (x, y, z) = x + y + z
(* val add_tuple3 : int * int * int -> int = <fun> *)
let () = print_int (add_tuple3 (3, 4, 5)); print_newline ()
(* 12 *)
```

- ▶ Donc on voit qu'on peut utiliser :
 - ▶ un pattern matching avec `match` et motif `(x, y)`
 - ▶ un pattern matching avec un `let` destructurant sur motif irréfutable
 - ▶ un pattern matching directement au niveau de l'argument de la fonction

Tuples en OCaml

- ▶ Si `t1` et `t2` sont des types, `t1 * t2` est le type des couples dont le premier élément est de type `t1` et le deuxième de type `t2`.
- ▶ De même pour un triplet `t1 * t2 * t3`, etc
- ▶ Les couples, triplets, etc s'écrivent avec la syntaxe `(x, y)`, `(x, y, z)`, etc
- ▶ On peut faire du pattern matching sur un couple, un triplet, etc,
 - ▶ avec `match`
 - ▶ avec `let`
 - ▶ directement dans l'argument d'une fonction

Tuples en Rocq

- ▶ En Rocq, les tuples ne sont pas primitifs.
- ▶ Dans le prélude (ce qui est chargé par défaut au démarrage de Rocq) on a `prod` :

```
Inductive prod (A B : Type) : Type := pair (a : A) (b : B).
```

- ▶ la notation pour les types :

```
Notation "x * y" :=  
  (prod x y) (at level 40, left associativity) : type_scope
```

- ▶ une notation pour enchaîner les paires :

```
Notation "( x , y , .. , z )" :=  
  (pair .. (pair x y) .. z) : core_scope.
```

- ▶ `Check (nat * bool)%type. (* (nat * bool)%type : Set *)`
`Check (3, true, 1). (* (3, true, 1) : nat * bool * nat *)`

Tuple en Rocq (2)

```
(* définitions équivalentes ... *)  
(* "vrai" add_tuple2, sans notation : *)  
Definition add_tuple2_match c := match c with (pair x y) => x + y end.  
Definition add_tuple2 c := match c with (x, y) => x + y end.  
(* motif irréfutable, le constructeur est noté "(_, _)" *)  
Definition add_tuple2a c := let '(x, y) := c in x + y.  
(* motif irréfutable directement dans l'argument *)  
Definition add_tuple2b '(x, y) := x + y.
```

Tuple en Rocq (3)

```
(* "vraie" définition de add_tuple3 : *)
Definition add_tuple3_match1 t :=
  match t with (pair c z) =>
    match c with (pair x y) => x + y + z end end.
(* idem avec notation *)
Definition add_tuple3_match2 t :=
  match t with (c, z) =>
    match c with (x, y) => x + y + z end end.
(* pattern matching imbriqué (pair (pair x y) z) *)
Definition add_tuple3 t := match t with (x, y, z) => x + y + z end.
(* Motif irréfutable sur constructeurs *)
Definition add_tuple3a '(x, y, z) := x + y + z.
Fail Definition add_tuple3b (t : nat * nat * nat) :=
  let (x, y, z) := t in x + y + z.
Definition add_tuple3b (t : nat * nat * nat) :=
  let (c, z) := t in let (x, y) := c in x + y + z.
Print add_tuple3.
Print add_tuple3a. (* même chose *)
Print add_tuple3b. (* même chose *)
```

Tuples en Rocq : Bilan

- ▶ Type `prod` non primitif avec notations astucieuses :
- ▶ `t1 * t2` pour `prod t1 t2`
- ▶ `t1 * t2 * t3` pour `prod (prod t1 t2) t3`
- ▶ etc
- ▶ `(x, y)` pour `pair x y`
- ▶ `(x, y, z)` pour `pair (pair x y) z`
- ▶ etc

Tuples en Rocq : Bilan

- ▶ Type `prod` non primitif avec notations astucieuses :
- ▶ `t1 * t2` pour `prod t1 t2`
- ▶ `t1 * t2 * t3` pour `prod (prod t1 t2) t3`
- ▶ etc
- ▶ `(x, y)` pour `pair x y`
- ▶ `(x, y, z)` pour `pair (pair x y) z`
- ▶ etc
- ▶ Remarque : l'équivalent de `prod` pour des propositions (c'est-à-dire des types vivant dans `Prop`) est `and` et l'équivalent de `pair` est `conj`.

Fonction de plusieurs variables avec `tuple`s

Que ce soit en OCaml ou en Rocq, les fonctions qui prennent un argument de type `tuple` sont :

- ▶ un peu plus lourdes à définir (pattern matching, d'une façon ou d'une autre sur le tuple)
- ▶ un peu plus lourdes à utiliser (parenthèses et virgules)
- ▶ En général, on préfère les fonctions curryfiées (prochaine partie), mais les tuples sont indispensables comme type de retour, ...
- ▶ tout comme les propositions formées avec `/\` sont indispensables dans les conclusions des implications.

Types de fonctions en Rocq et en OCaml

Tuples en OCaml et en Rocq

Curryfication

Plusieurs flèches dans `Prop`

`False` et négation

Retourner une fonction

- ▶ En OCaml (dans utop :)
(+);;
- : int -> int -> int = <fun>
- ▶ L'opérateur -> est associatif à droite.
- ▶ Donc l'addition est de type `int -> (int -> int)`.
- ▶ # (+) 2;;
- : int -> int = <fun>
- ▶ La valeur `( + ) 2` est la fonction qui à `x` associe `2 + x`.
- ▶ La valeur `( + ) 2 5` est l'application de cette dernière fonction à 5, donc `2 + 5`, c'est-à-dire 7.

Retourner une fonction

- ▶ On peut définir nous-même une fonction qui retourne une fonction

```
let foo = fun x -> (fun y -> x + 2 * y)
```

- ▶ On n'a pas besoin des parenthèses :

```
let foo = fun x -> fun y -> x + 2 * y
```

- ▶ Sucre syntaxique 1 :

```
let foo = fun x y -> x + 2 * y
```

- ▶ 100% pur sucre syntaxique :

```
let foo x y = x + 2 * y
```

- ▶ Mais c'est bien la première forme qui est « la vraie » définition.

Fonctions partielles

- ▶ On a gratuitement la fonction partielle suivant y

```
let bar = foo 42  
(* bar : int -> int *)
```

- ▶ pour l'autre fonction partielle, il faut l'écrire nous mêmes par contre

```
let baz x = foo x 42
```

- ▶ Ou bien, comme fonction anonyme :

```
fun x -> foo x 42
```

En Rocq

- C'est pareil :

```
Definition foo1 := fun x => (fun y => x + 2 * y).
```

```
Definition foo2 := fun x => fun y => x + 2 * y.
```

```
Definition foo3 := fun x y => x + 2 * y.
```

```
Definition foo4 x y := x + 2 * y.
```

- Idem pour les applications partielles.

```
Definition bar := foo 42. (* bar : nat -> nat *)
```

```
Check (fun z => foo z 42). (* nat -> nat *)
```

Conclusion

- ▶ Si t_1 , t_2 et t_3 sont des types, le type $t_1 \rightarrow t_2 \rightarrow t_3$ signifie en fait $t_1 \rightarrow (t_2 \rightarrow t_3)$ et est donc le type des fonctions qui à chaque argument de type t_1 associe une valeur qui est une fonction de t_2 vers t_3 .
- ▶ Une fonction « de deux variables » est en fait une telle fonction.
- ▶ Il y a plein de sucre syntaxique pour nous le faire oublier.
- ▶ Une fonction « de plusieurs variables » écrite sous cette forme est dite « curryfiée » en hommage à Haskell Curry (1900 - 1982), logicien américain.

Types de fonctions en Rocq et en OCaml

Tuples en OCaml et en Rocq

Curryfication

Plusieurs flèches dans **Prop**

False et négation

$A \rightarrow B \rightarrow C$

- ▶ Tout ce qui a été dit précédemment est encore vrai dans **Prop**.
- ▶ On peut interpréter un terme $h : A \rightarrow B \rightarrow C$ comme :
 - ▶ une fonction qui prend un élément de type A et donne une fonction de type $B \rightarrow C$
 - ▶ une fonction qui prend un élément de type A et un élément de type B et donne un élément de type C
 - ▶ une preuve d'une implication $A \rightarrow (B \rightarrow C)$
 - ▶ une preuve d'une formule disant intuitivement, si je sais prouver A puis B , alors je sais prouver C .

Exemple 1

```
Lemma foo (A B : Prop) : A -> (B -> A).
```

```
Proof.
```

```
  (* exact (fun (a : A) => fun (_ : B) => a). *)
```

```
  exact (fun a _ => a).
```

```
Qed.
```

Avec des tactiques :

```
Lemma foo (A B : Prop) : A -> (B -> A).
```

```
Proof.
```

```
  intros hA _. exact hA.
```

```
Qed.
```

Exemple 2

Utiliser une double implication, avec des tactiques :

Lemma baz1 (A B C : **Prop**) : A -> B -> (A -> B -> C) -> C.

Proof.

```
  intros hA. (* on suppose hA : A *)
  intros hB. (* on suppose hB : B *)
  intros hABC. (* on suppose hABC : A -> B -> C *)
  apply hABC. (* génère deux buts ! *)
- exact hA.
- exact hB.
```

Qed.

Exemple2 avec terme de preuve

```
Lemma baz2 (A B C : Prop) : A -> B -> (A -> B -> C) -> C.
```

```
Proof.
```

```
  exact (fun hA hB hABC => hABC hA hB).
```

```
Qed.
```

C'est en fait exactement la même preuve. On peut s'en convaincre avec :

- ▶ la commande vernaculaire `Show Proof` à la fin de la preuve
- ▶ ou la commande `Print baz1`.

Plusieurs flèches dans Prop

- ▶ Comme avec les fonctions curryfiées qu'on peut voir comme des fonctions de plusieurs variables...
- ▶ on peut interpréter une formule du type $A \rightarrow B \rightarrow C$
- ▶ (c'est-à-dire, avec les parenthèses cachées, $A \rightarrow (B \rightarrow C)$)
- ▶ comme :

Si A et B, alors C.

- ▶ Et comme pour les fonctions, on préfère cette écriture (curryfiée) à celle vue la semaine dernière :
 $A \wedge B \rightarrow C$
- ▶ Mais la conjonction reste indispensable dans certains cas (en conclusion, notamment).

Types de fonctions en Rocq et en OCaml

Tuples en OCaml et en Rocq

Curryfication

Plusieurs flèches dans Prop

False et négation

Proposition False

- ▶ On a déjà vu :

Inductive True : **Prop** := I.

Autrement dit, True a un seul constructeur qui n'a besoin de rien, on peut toujours prouver True.

- ▶ Pour False, c'est le contraire :

Inductive False : **Prop** := .

Il n'y a aucun constructeur, on ne peut pas contruire de terme de type False (heureusement!).

- ▶ Penser à l'ensemble vide.

Ex Falso Quod Libet

- ▶ De faux on peut prouver n'importe quoi.
- ▶ `Lemma foo (A : Prop) : False -> A.`
`Proof.`
 `intros hF.`
 `destruct hF. (* termine la preuve *)`
`Qed.`
- ▶ Avoir `False` dans ses hypothèses correspond à un cas impossible, qu'on peut donc évacuer rapidement.